

Unit no -5 Compiler

5.1 Compiler

5.2 Phases of compiler-lexical analysis, syntax analysis, semantic analysis, Intermediate code generation, code optimization code generation.

Interpreter

Difference between Compiler and interpreter

Compiler design option:-

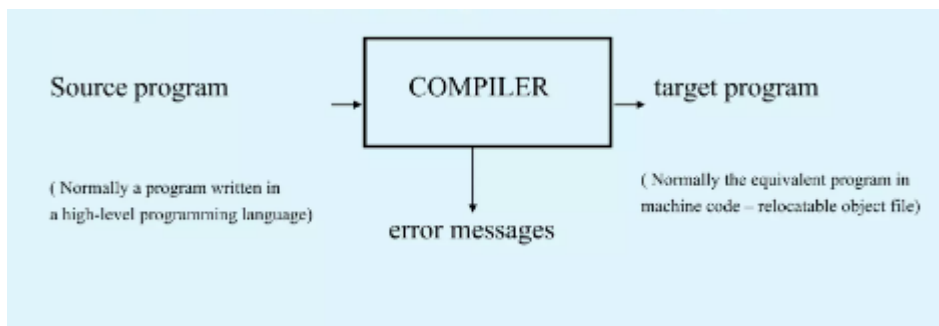
1. Bootstrap Compiler
2. Cross Compiler
3. Compiler Compiler
4. Incremental Compiler

Compiler:-

A compiler is program takes a program written in source language and translate it into an equivalent program in a target language. An important role of the compiler is to report any errors in source program that it defects during the translated process.

The object program generated by compiler is machine dependent It means program compiled for one type of machine will not run in another type. Therefore every type of machine must have it personal compiler for particular language.

Example:-Turbo C



Role of Compiler:-

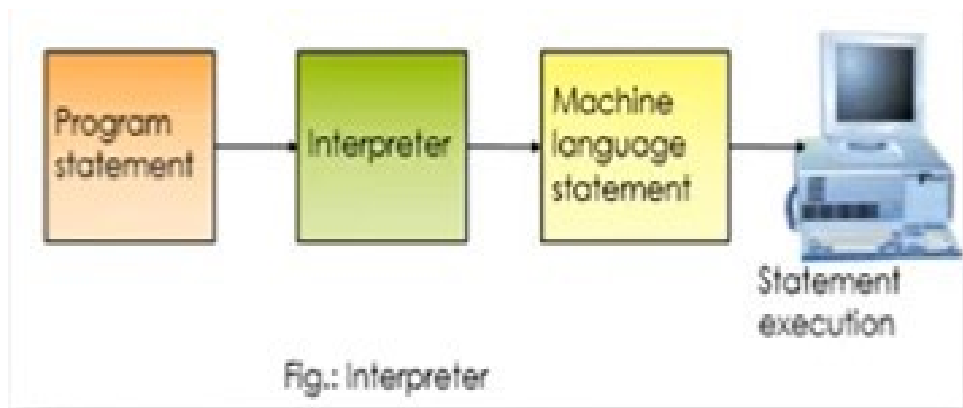
- Compilers reads the source code, outputs executable code
- Translates software written in a higher-level language into instructions that computer can understand. It converts the text that a programmer writes into a format the CPU can understand.
- The process of compilation is relatively complicated. It spends a lot of time analyzing and processing the program.
- The executable result is some form of machine-specific binary code.

Interpreter:

An interpreter accepts the source program and perform the actions associated with the instruction

It create virtual execution environment

An interpreter reads source code one instruction or line at a time converts this into machine code or some intermediate form and execute it.



Role of Interpreter:-

- The interpreter converts the source code line-by-line during RUN Time.
- Interpret completely translates a program written in a high-level language into machine level language.
- Interpreter allows evaluation and modification of the program while it is executing.
- Relatively less time spent for analyzing and processing the program
- Program execution is relatively slow compared to compiler

Differentiate between Compiler and Interpreter

Compiler	Interpreter
The compiler translate the entire program in one go and then execute it	The Interpreter performs this operation line by line i.e takes one statement then translates it and execute it and then takes another statement
Thus, a compiler converts the high level instruction into machine language.	The interpreter converts the high level instruction into an intermediate form
Compiler generates the list of error after it has completed the compilation process.	An interpreter stops translating after it gets the first error.
Compiler takes a larger amount of time in analyzing and processing the high level languages code.	An Interpreter takes lesser time in the same process.
The overall time of a code is faster for compiler It creates executable file that runs directly on CPU	The overall execution time of the code is slower as it interpreter code one line as time

Debugging is more difficult one error can produce many spurious errors.	Debugging is easier each line of code is analyzed and checked before being executed.
A computer is more likely to crash the computer as the machine code is running directly on the CPU	An interpreter is less likely to crash as the instructions are being carried out either on the interpreter's command line or within the virtual machine environment which is protecting the computer from being directly accessed by code
Uses more memory; all the execution code needs to be loaded into memory.	Uses less memory; source code only has to be present one line at a time in memory.
Unauthorized modification to the code is more difficult the executable is in the form of machine code so it is difficult to understand program flow.	Easier to modify as the instructions are high level As a result the program flow is easier to understand and modify.
An independent executable file is created by the computer.	Interpreter is required by an interpreter program each time

Language Processing System:-

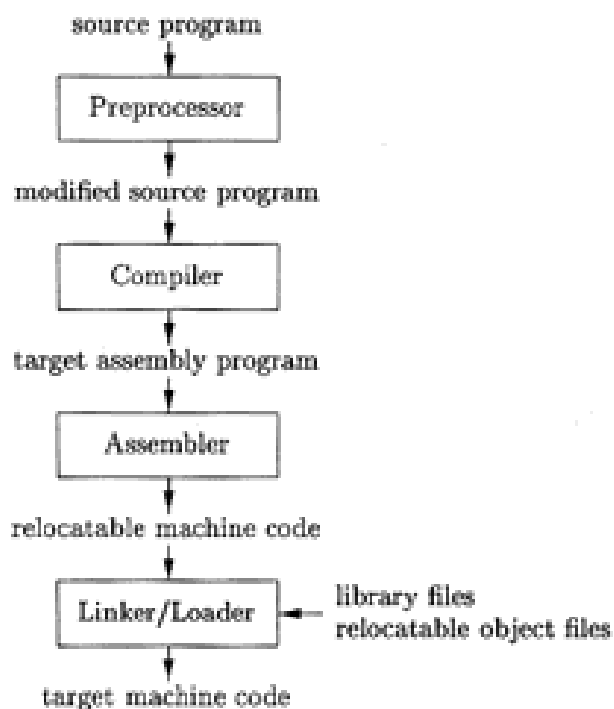


Figure 1.5: A language-processing system

Pre Processor :-

- Modules are stored in separate files.
- Source program is collected by a separate program.

Compiler:-

- Source program is fed to a compiler.
- Output- assembly-language program.
- Assembly language is easier to produce as output.
- Easier to debug.

Assembler:-

- Assembly language into relocatable machine code.
- Large programs are compiled into small modules.
- The relocatable machine code have to be linked together with other relocatable object files and library files.
- This code actually runs on the machine.

Linker:-

- Resolves external memory addresses,
- The code in one file may refer to a location in another file.

Loader:-

- Puts together all of the executable object files into memory for execution.

Phases of Compiler:-

Q.(Draw diagram for the phase of Compiler)

Q.(Explain phases of compiler with diagram.)

Each phase transforms the source program form one representation into another representation

Phases of Compiler

1. Lexical analysis

- Its main task is to read the source program and if the elements of the program are correct it generates as output a sequence of tokens that the parser uses for syntax analysis.
- It recognizes keywords, operators and identifiers, integers, floating point numbers, character strings and other similar items that form the source program.

2. Syntax Analysis

- In this phase the compiler must recognize the phases (syntactic construction); each phrase is a semantic entry and is a string of tokens that has meaning, and 2nd Interpret the meaning of the constructions.
- Syntactic analysis also notes syntax errors and assure some sort of recovery.

1. Semantic Analysis

- The semantic analyzer uses the syntax tree and the information in symbol table to check the source program for semantic consistency with language definition.

2. Intermediate code Generation:-

- This phase is typically a routine that are called when a construct is recognized. The purpose of these routines is to on intermediate form of source program and adds information to identifier table.

3. Code Optimization

- Two types of optimization is performed by compiler, machine dependent and machine independent. Machine dependent optimization is so intimately related to instruction that the generated. It was incorporated into the code generation phase.
- Where Machine independent optimization is was done in a separate optimization phase.

4. Code generation.

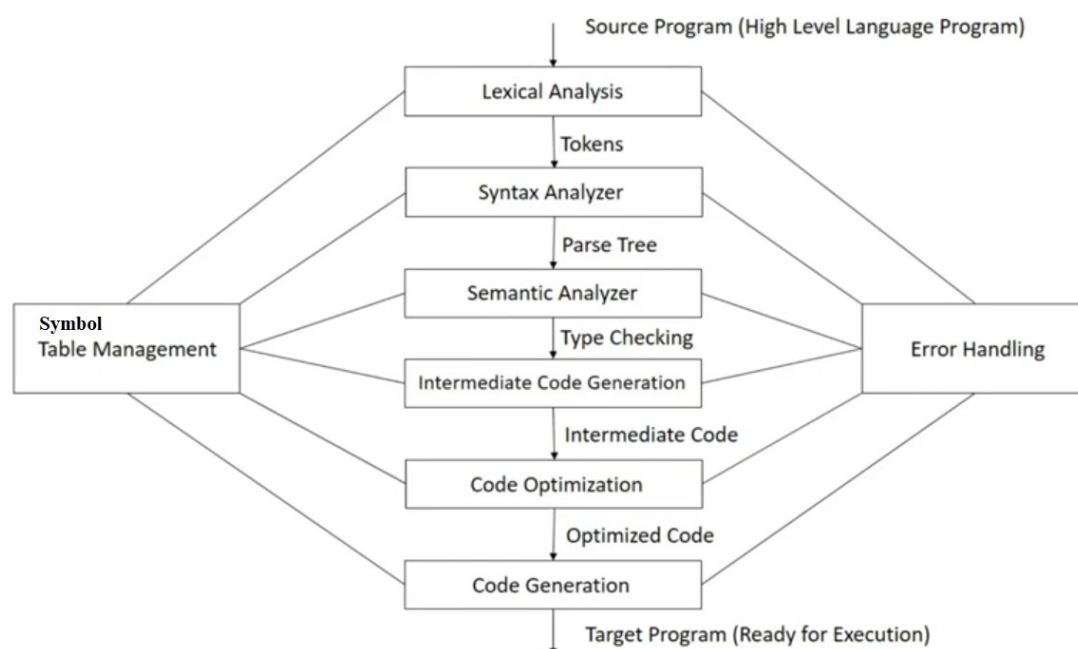
- This phase produce a program which can be in Assembly or machine language.
- This phase has matrix as input.
- It uses the code production in the matrix to produce code.

• Symbol-Table Management:-

- Data structure containing a record for each variable name, with fields for the attributes of the name. Designed to allow the compile. Find the record for each name quickly and to store or retrieve data from that record quickly.

• Error Handling:-

- Each phase can encounter errors.
- After detecting an error, a phase must somehow deal with that error, so that compilation can proceed, allowing further errors in the source program to be detected.
- A compiler that stops when it finds the first error is not as helpful as it could be.
- The syntax and semantic analysis phases usually handle a large fraction of the errors detectable by the compiler.
- The lexical phase can detect errors where the characters remaining in the input do not form any token of the language.



The structure of compiler:-

It is divided into 2 parts:-

1. Analysis Phase(machine dependent):-

In this phase an intermediate representation is created from the given source program.

Parts of this phase are:-

- Lexical analyzer
- Syntax analyzer
- Semantic analyzer

2. Synthesis Phase(machine Independent):-

In this phase equivalent target program is created from this intermediate representation.

Parts of this phase:-

- Intermediate code generation
- Code optimization
- Code generation

- The analysis part breaks up the source program into constituent pieces and imposes a grammatical structure on them. It then uses this structure to create an intermediate representation of the source program.

- If the analysis part detects that the source program is either syntactically ill formed or semantically unsound, then it must provide informative messages, so the user can take corrective action.

- The analysis part also collects information about the source program and stores it in a data structure called a symbol table, which is passed along with the intermediate representation to the synthesis part.

- The synthesis part constructs the desired target program from the intermediate representation and the information in the symbol table. The analysis part is often called the front end of the compiler; the synthesis part is the back end.

1. Lexical Analysis:-

- It reads the source program one character at a time and group them into tokens.
- A tokens is a sequences of characters with collective meaning e.g. tokens are- keywords, identifiers, operators, labels, constant, operator, delimiters
- They are put in symbol table
- Tokens are in pair of <token name, attribute value>
- Tokens are
Keywords are:if,else,for,while
Operators:- <,>,<=,>=,+,-,* etc
Constant:- 10,20,30.4,50.0 etc
Identifier:- Goto 200
- Non tokens are:- white space,comments

Example:- newval= oldval+12;

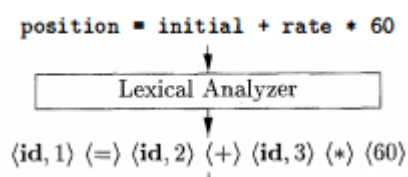
Token type	Value
------------	-------

Identifiers	Newval,oldval
Operators	=,+
Constants	12

Example:- $a=b+5*c-10.5$

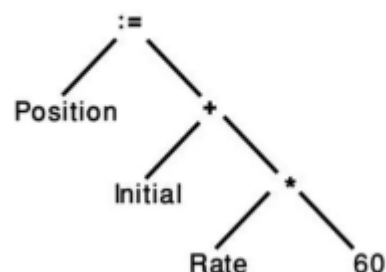
Token type	Value
Identifiers	a,b,c
Operators	=,+,*,-
Constants	5,10.5

- Example:- `position=initial+rate*60;`



2. Syntax analysis:-

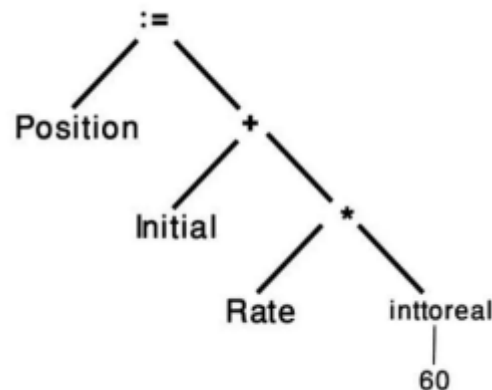
- A syntax analyzer create the syntactic structure (generally parse tree of given program)
- It also call a parser
- A typical representation is a syntax tree in which each interior node represents an operation and the child of node represent the argument of the operation.
- The syntax of a language is specified by a context free grammar (CFG). The rules in a CFG are mostly recursive.
- A syntax analyzer checks whether a given program satisfies the rules implied by a CFG or not.
- If it satisfies, the syntax analyzer creates a parse tree for the given program.
- Ex: We use BNF (Backus Naur Form) to specify a CFG
- Example:- `position=initial+rate*60;`



3. Semantic analysis:-

- A semantic analyzer checks the source program for semantic errors and collects the type information for the code generation.

- Type-checking is an important part of semantic analyzer. Normally semantic information cannot be represented by a context-free language used in syntax analyzers.
- Context-free grammars used in the syntax analysis are integrated with Context-free grammars used in the syntax analysis are integrated with attributes (semantic rules)
 - the result is a syntax-directed translation, ◦ Attribute grammars
- Example:- `position=initial+rate*60;`



4. Intermediate Code Generation

- In the process of translating a source program into target code a compiler may construct one or more intermediate representation which can have a variety of forms.
- Syntax tree are a form of intermediate representation they are commonly used during syntax and semantic analysis.
- Compiler produce or generate an explicit low level or machine like intermediate representation which we can think of as a program for an abstract machine.
- This intermediate representation should have 2 important properties
 - a. it should be easy to produce
 - b. It should be easy to translate into the target machine.
- c. Example:- `position=initial+rate*60;`

```

Temp1  :=  inttoreal (60)
Temp2  :=  id3 * temp1
Temp3  :=  id2 + temp2
id1 :=  temp3
  
```

5. Code Optimization:-

Two types of code optimization

1. Machine dependent:-

When a subexpression occurs in a same statement more than one we can delete all duplicate matrix entries and modify all references to the deleted entry so that they refer to the remaining copy of that subexpression
Optimization done during code generation phase.

2. Machine Independent:-

It is so intimately related to instruction that generated.

It was taken into code generation phase whereas machine independent optimization was done in a separate optimization phase

Database used by optimization

Matrix:

Operator	Operand1	Operand2	Forward Pointer	Backward Pointer
----------	----------	----------	-----------------	------------------

This is the major database used by the optimization phase.

For insertion and deletion of entries into the matrix, the chaining information is added to each entry, forward and backward pointers.

This avoids necessity of reordering and relocating matrix entries when an entry is added or deleted.

The forward pointer is the index of next matrix entry and allow the code generation phase to go through the matrix in the proper order.

The backward pointer is the index of previous matrix entry and allows sequencing through the matrix as may be needed by the optimization technique.

Four technique used by Compiler during m/c independent optimization:-

1. Elimination of common subexpression
2. Compile time compute
3. Boolean expression optimization
4. Move invariant computation outside of loop.

1. **Elimination of common subexpression**

Elimination of common sub expression Expressions which yield same value and must be in a same statement are common sub expressions.

e.g. Consider following statement. $B=A \ A=C * D * (D * C + B)$

The elimination algorithm follows following steps:

1. Place the matrix in a form so that common sub expressions can be recognized.
2. Recognize two sub expressions as being equivalent.
3. Eliminate one of them.
4. Alter the rest of matrix to reflect the elimination of this entry.

Source code:
 $B = A$
 $A = C * D * (D * C + B)$

Matrix After steps 1 & 2

M1	=	B	A	1	2
M2	*	C	D	2	3
M3	*	C	D	3	4
M4	+	B	M3	4	5
M5	*	M2	M4	5	6
M6	=	A	M5	6	?

Matrix before optimization

M1	=	B	A	1	2
M2	*	C	D	2	3
M3	*	D	C	3	4
M4	+	M3	B	4	5
M5	*	M2	5	5	6
M6	=	A	M5	6	?

Matrix After steps 3 & 4

M1	=	B	A	1	2
M2	*	C	D	2	4
M3	*	C	D	2	4
M4	+	B	M2	4	5
M5	*	M2	M4	5	6
M6	=	A	M5	6	?

2. Compile time compute(evaluation)

Compile time evaluation (Computation) certain computations in a program, involving constants can be performed during the compilation stage save both space and execution time.

The main optimization of this type is constant folding. If all the operands in an expression are constants, the operation can be performed at compile time itself. The result of the operation, itself a constant, then replaces the original expression.

e.g. $A = 2 * 150 / B$;
 Assignment of above type can be replaced by
 $A = 300 / B$

Before Optimization

M1	*	2	150
M2	/	M1	B
M3	=	A	M2

After Optimization

M1			
M2	/	300	B
M3	=	A	M2

3. Boolean expression optimization

Boolean expression optimization Properties of Boolean expression can be used to shorten their computations.

e.g. In a statement IF A OR B OR C THEN ...

When A, B, and C are expressions Rather than generating code that will test each of the expressions A, B, and C, only code is generated so that if A is computed as true, then B OR C is not computed, and similarly for B and C.

4. **Move invariant computation outside of loop.**

Move invariant computations outside of loops If a computation within a loop depends on a variable that does not change within that loop, the computations may be moved outside the loop. This involves three steps: 1. Recognition of invariant computations. 2. Discovering where to move the invariant computation. 3.

Moving the invariant computation.

- Example :-

Position =initial+rate *60

```
Temp1  :=  id3 * 60.0
```

```
id  :=  id2 + temp1
```

6. **Code Generation :-**

Code generation phase produces the object code by deciding on the memory location for data selecting code for each data item and selecting the register is to be done

- This phase produce a program which can be in Assembly or machine language.
- This phase has matrix as input.
- It uses the code production in the matrix to produce code.
- Example :-

Position =initial+rate *60

```
MOVF  id3, r2
```

```
MULF  #60.0, r2
```

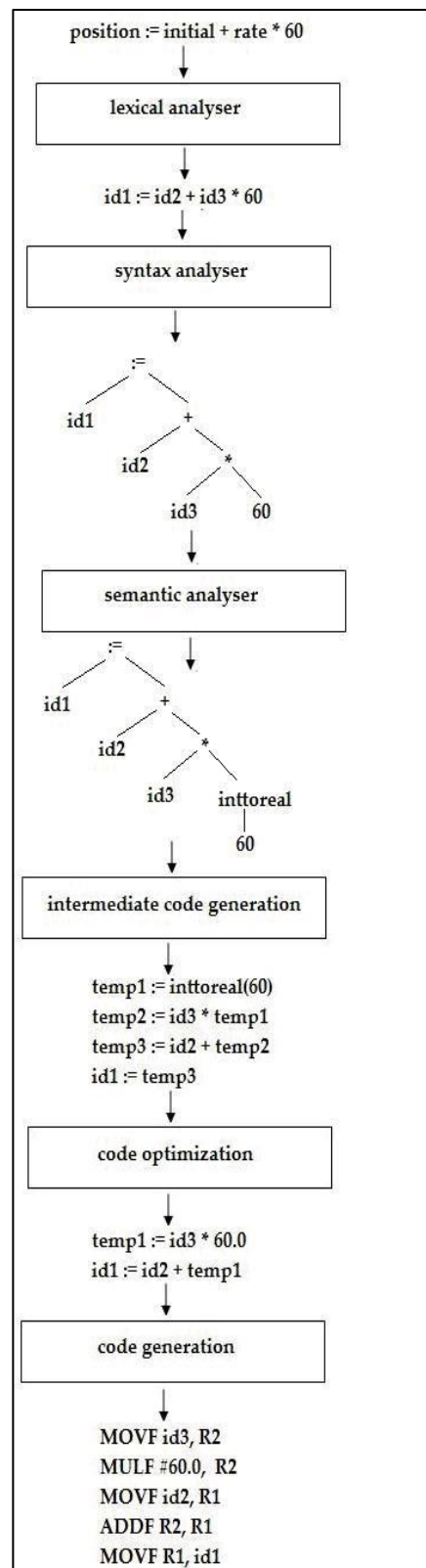
```
MOVF  id2, r1
```

```
ADDF  r2, r1
```

```
MOVF  r1, id1
```

Example :-

Position =initial+rate *60



Compiler Design Option:-

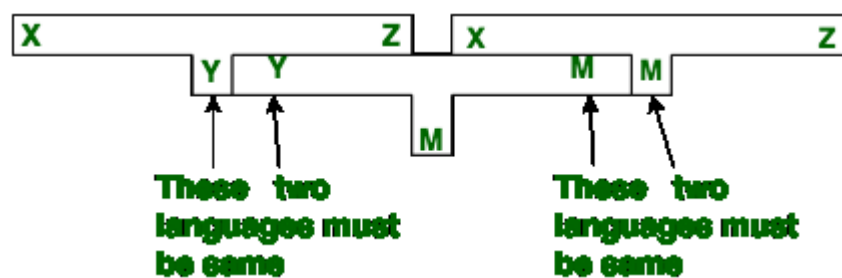
1. Bootstrap Compiler

Bootstrapping is a process in which simple language is used to translate more complicated program which in turn may handle for more complicated program. This complicated program can further handle even more complicated program and so on.

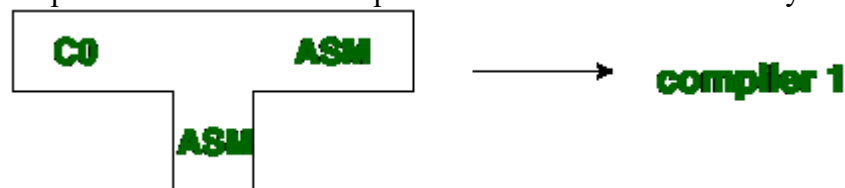
Writing a compiler for any high level language is a complicated process. It takes lot of time to write a compiler from scratch. Hence simple language is used to generate target code in some stages. To clearly understand the Bootstrapping technique consider a following scenario.

Suppose we want to write a cross compiler for new language X. The implementation language of this compiler is say Y and the target code being generated is in language Z. That is, we create XYZ. Now if existing compiler Y runs on machine M and generates code for M then it is denoted as YMM. Now if we run XYZ using YMM then we get a compiler XMZ. That means a compiler for source language X that generates a target code in language Z and which runs on machine M.

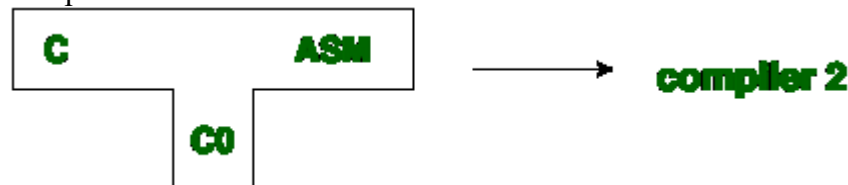
Compiler which takes C language and generates an assembly language as an output with the availability of a machine of assembly language.



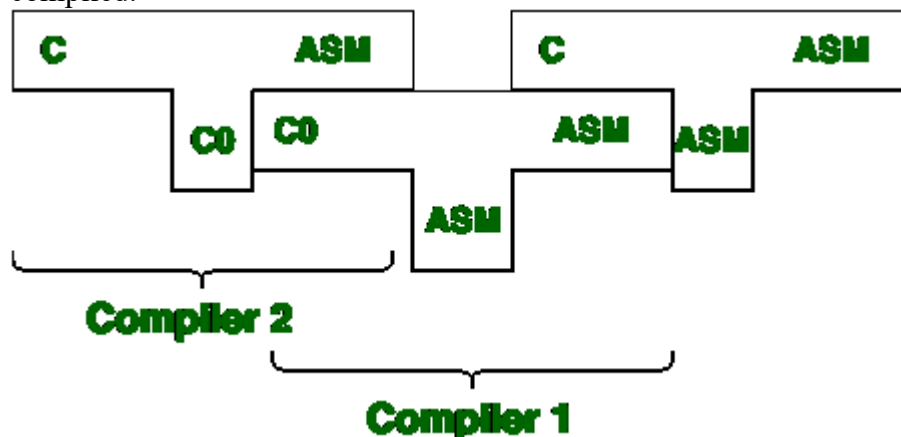
Step-1: First we write a compiler for a small of C in assembly language



Step-2: Then using with small subset of C i.e. C0, for the source language c the compiler is written.



Step-3: Finally we compile the second compiler. using compiler 1 the compiler 2 is compiled.



Step-4: Thus we get a compiler written in ASM which compiles C and generates code in ASM.

2. Cross Compiler

A cross compiler is a compiler capable of creating executable code for a platform other than the one on which the compiler is running. For example, a cross compiler executes on machine X and produces machine code for machine Y.

Where is the cross compiler used?

- In bootstrapping, a cross-compiler is used for transitioning to a new platform. When developing software for a new platform, a cross-compiler is used to compile necessary tools such as the operating system and a native compiler.
- For microcontrollers, we use cross compiler because it doesn't support an operating system.
- It is useful for embedded computers which are with limited computing resources.
- To compile for a platform where it is not practical to do the compiling, a cross-compiler is used.
- When direct compilation on the target platform is not infeasible, so we can use the cross compiler.
- It helps to keep the target environment separate from the built environment.

T-Diagram for Cross Compiler



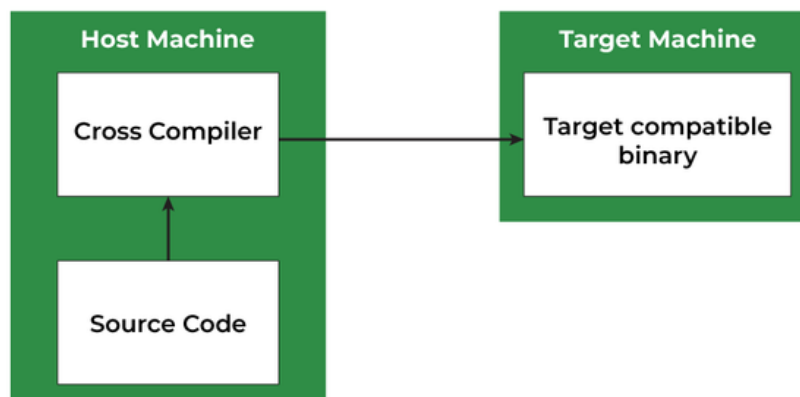
A compiler is characterized by three languages:

1. The source language that is compiled.
2. The target language T is generated.
3. The implementation language that is used for writing the compiler.

How does cross-compilation work?

A cross compiler is a compiler capable of creating executable code for a platform other than the one on which the compiler is running. In Para virtualization, one computer runs multiple operating systems and a cross compiler could generate an executable for each of them from one main source.

Cross Compiler Operation



How compiler is different from a cross-compiler?

The native compiler is a compiler that generates code for the same platform on which it runs and on the other hand, a Cross compiler is a compiler that generates executable code for a platform other than one on which the compiler is running. Check out this article for more information

3. Compiler compiler or compiler generator:-

It is a programming tool that creates a parser, interpreter or compiler from some form of formal description of programming language and machine.

Common type compiler compiler is parser generator. It only handles syntactic analysis.

The input of a parser generator is a grammar file typically written in Backus Naur form (BNF) or extended Backus Naur form (EBNF) that defines the syntax of target programming language.

4. Incremental compiler

Incremental Compiler is a compiler that generates code for a statement, or group of statements, which is independent of the code generated for other statements.

Examples :

C/C++ GNU Compiler, Java eclipse platform, etc.

The Incremental Compiler is such a compilation scheme in which only modified source text gets recompiled and merged with previously compiled code to form a new target code. Thus incremental compiler avoid recompilation of the whole source code on some modification. Rather only modified portion of source program gets compiled.

Code --> Compilation

New code --> New Compilation

(Previous (Previous Compilation +

Code + Changes) compilation of modified part)

Need of Incremental Compiler :

Much of a programmer's time is spent in an edit-compile-debug workflow as following.

- you make a small change (often in a single module or even function).
- you let the compiler translate the code into a binary, and finally.
- you run the program or a bunch of unit tests in order to see results of the change.

How can this concept (concept of incremented compilation) be implemented :

We have already heard that computing something incrementally means updating only those parts of the computation's output that need to be adapted in response to a given change in the computation's inputs. One basic strategy we can employ to achieve this is to view one big computation (like compiling a program) as a composite of many smaller, interrelated computations that build upon each other. Each of those smaller computations will yield an intermediate result that can be cached and hopefully re-used in a later iteration, sparing us the need to re-compute that particular intermediate result again.

Features of incremental compiler :

- During program development process modifications to Source program can cause recompilation of whole source text. This overhead is reduced in incremental compiler.
- Run time errors can be patched up just like compile time errors by keeping program modification as it is.
- The compilation process is faster.
- The memory used is less.
- Handling batch programs become very flexible using Incremental compiler.
- In incremental compiler program structure table is maintained for memory allocation of target code. When a new statement is compiled the new entry for it is created in program structure table. Thus memory allocation required for incremental compiler need not be contiguous.
- Helps in tracking the dependencies on source program.